The SQL Indexes

SQL Indexes are special lookup tables that are used to speed up the process of data retrieval. They hold pointers that refer to the data stored in a database, which makes it easier to locate the required data records in a database table.

SQL Indexes work similar to the index of a book or a journal.

While an index speeds up the performance of data retrieval queries (SELECT statement), **it slows down the performance of data input queries (UPDATE and INSERT statements).** However, these indexes do not have any effect on the data.

SQL Indexes need their own storage space within the database. Despite that, the users cannot view them physically as they are just performance tools.

The CREATE INDEX Statement

An index in SQL can be created using the **CREATE INDEX** statement. *This statement allows you to name the index, to specify the table and which column or columns to index, and to indicate whether the index is in an ascending or descending order.*

Preferably, an index must be created on column(s) of a large table that are frequently queried for data retrieval.

Syntax

The basic syntax of a **CREATE INDEX** is as follows –

```
CREATE INDEX index_name ON table_name;
```

Types of Indexes

There are various types of indexes that can be created using the CREATE INDEX statement. They are:

- **Unique Index**
- **Single-Column Index**
- **Composite Index**
- **Implicit Index**

Unique Indexes

Unique indexes are used not only for performance, but also for data integrity. A **unique index does not allow any duplicate values** to be inserted into the table. **It is automatically created by PRIMARY and UNIQUE constraints when they are applied on a database table,** in order to prevent the user from inserting duplicate values into the indexed table column(s).

The basic syntax is as follows.

CREATE UNIQUE INDEX index_name on table_name (column_name);

Single-Column Indexes

A single-column index is created only on one table column.

The syntax is as follows.

CREATE INDEX index_name ON table_name (column_name);

Composite Indexes

A composite index is an index that can be created on two or more columns of a table.

Its basic syntax is as follows.

CREATE INDEX index_name on table_name (column1, column2);

Implicit Indexes

Implicit indexes are indexes that are automatically created by the database server when an object is created. For example, indexes are automatically created when primary key and unique constraints are created on a table in MySQL database.

The DROP INDEX Statement

*An index can be dropped using SQL **DROP** command. Dropping an index can effect the query performance in a database. Thus, an index needs to be dropped only when it is absolutely necessary.*

The basic syntax is as follows −

DROP INDEX index_name;

When should indexes be avoided?

Although indexes are intended to enhance a database's performance, there are times when they should be avoided.

The following guidelines indicate when the use of an index should be reconsidered.

- *Indexes should not be used on small tables.*
- *They should not be used on tables that have frequent, large batch updates or insert operations.*
- *Indexes should not be used on columns that contain a high number of NULL values.*
- *Columns that are frequently manipulated should not be indexed.*

What is SQL Index?

An **SQL index** is a special lookup table that helps in efficiently searching or querying database tables to retrieve required data. For example, when we try to retrieve data from multiple tables using joins, indexes improve the query performance.

Indexes are used to optimize the query performance of any Relational Database Management System (RDBMS) as data volumes grow. Hence, they are preferred to be used on frequently queried large database tables.

Creating an SQL Index

An index can be created on one or more columns of a table in an SQL database using the **CREATE INDEX** statement.

Syntax

Following is the syntax of **CREATE INDEX** statement in SQL –

```
CREATE INDEX index_name
ON table_name (column_name1, column_name2,... column_nameN);
```

Here,

- **index_name** This specifies the name of the index that you want to create.
- **table_name** This specifies the name of the table on which you want to create the index.
- **(column_name1, column_name2...column_nameN)** are the names of one or more columns on which the index is being created.

Example

To create an index on a database table, we first need to create a table. So, in this example, we are creating a table named **CUSTOMERS** using the following query

```
CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR(15) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS VARCHAR(25),
  SALARY DECIMAL(10, 4),
  PRIMARY KEY(ID));
);
```

Then, insert some values into the CUSTOMERS table using the following query −

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', '32', 'Ahmedabad', 2000),

(2, 'Khilan', '25', 'Delhi', 1500),

(3, 'Kaushik', '23', 'Kota', 2000),

(4, 'Chaitali', '25', 'Mumbai', 6500),

(5, 'Hardik','27', 'Bhopal', 8500),

(6, 'Komal', '22', 'Hyderabad', 9000),

(7, 'Muffy', '24', 'Indore', 5500);

Once the table is created, create an index for the column named **NAME** in the CUSTOMERS table using the following query −

```
CREATE INDEX index_name ON CUSTOMERS(NAME);
```

Output

When we execute the above query, the output is obtained as follows −

Query OK, 0 rows affected (0.06 sec)

Records: 0 Duplicates: 0 Warnings: 0

Verification

The following SHOW INDEX query is used to display all the indexes created on an existing table.

SHOW INDEX FROM CUSTOMERS;

In the list obtained, you can find the column name NAME, along with the ID in the list of indexes.

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	1	index_name	1	NAME

Creating an Index on Multiple Fields

We can also create an index on multiple fields (or columns) of a table using the CREATE INDEX statement. To do so, you just need to pass the name of the columns (you need to create the index on).

Example

Instead of creating a new table, let us consider the previously created CUSTOMERS table. Here, we are creating an index on the columns **NAME** and **AGE** using the following query −

CREATE INDEX mult_index_data on CUSTOMERS(NAME, AGE);

Output

When we execute the above query, the output is obtained as follows −

Query OK, 0 rows affected (0.04 sec)

Records: 0 Duplicates: 0 Warnings: 0

Verification

Now, let us list all the indexes that are created on the CUSTOMERS table using the following SHOW INDEX query –

SHOW INDEX FROM CUSTOMERS;

As you observe, you can find the column names NAME, and AGE along with ID (PRIMARY KEY), in the list of indexes.

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	1	index_name	1	NAME
customers	1	mult_index_data	1	NAME
customers	1	mult_index_data	2	AGE

SQL - Drop Index

The DROP statement in SQL is used to remove or delete an existing database object such as a table, index, view, or procedure. Whenever we use DROP statement with any of the database objects, it will remove them permanently along with their associated data.

And when that database object is an index, the **DROP INDEX** statement in SQL is used.

Dropping an SQL Index

An SQL Index can be dropped from a database table using the DROP INDEX statement.

It is important to understand that dropping an index can have a significant impact on the performance of your database queries. ***Therefore, only try to remove an index if you are sure that it is no longer required.***

Note − We cannot delete the indexes created by PRIMARY KEY or UNIQUE constraints. In order to delete them, you need to drop the constraints entirely using ALTER TABLE statement.

Syntax

Following is the syntax of the **DROP INDEX** command in SQL −

DROP INDEX index_name ON table_name;

Here,

- **index_name** is the name of the index that you want to drop.
- **table_name** is the name of the table that the index is associated with.

Example

In this example, we will learn how to drop an index on a table named **CUSTOMERS**, which can be created using the following query −

```
CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR(15) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS VARCHAR(25),
  SALARY DECIMAL(10, 4),
  PRIMARY KEY(ID));
);
```

Now, insert some values into the above created table using the following query −

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', '32', 'Ahmedabad', 2000),
(2, 'Khilan', '25', 'Delhi', 1500),
(3, 'Kaushik', '23', 'Kota', 2000),
(4, 'Chaitali', '25', 'Mumbai', 6500),
```

(5, 'Hardik','27', 'Bhopal', 8500),

(6, 'Komal', '22', 'Hyderabad', 9000),

(7, 'Muffy', '24', 'Indore', 5500);

Once the table is created, create an index on the column **NAME** in the CUSTOMERS table using the following query −

CREATE INDEX INDEX_NAME on CUSTOMERS(NAME);

Now, verify if the index is created on the CUSTOMERS table using the following SHOW INDEX query −

SHOW INDEX FROM CUSTOMERS;

On executing the above query, the index list is displayed as follows −

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	1	index_name	1	NAME

Then, drop the same index INDEX_NAME in the CUSTOMERS table using the following **DROP INDEX** statement −

DROP INDEX INDEX_NAME ON CUSTOMERS;

Output

If we compile and run the above query, the result is produced as follows −

Query OK, 0 rows affected (0.02 sec)

Records: 0 Duplicates: 0 Warnings: 0

Verification

Verify if the index for the column NAME is dropped using the following query −

SHOW INDEX FROM CUSTOMERS;

In the following list of indexes, you can observe that name of the column **Name** is missing.

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID

DROP INDEX with IF EXISTS

The DROP INDEX IF EXISTS statement in SQL is used to drop an index only if it exists in the table. This statement is specifically useful when you want to drop an index, but you are not sure if the index exists. This clause is **not** supported by MySQL.

The **IF EXISTS** clause ensures that the statement only removes the index if it exists. If the index does not exist, it simply terminates the execution.

Syntax

Following is the syntax of the DROP INDEX IF EXISTS in SQL −

DROP INDEX IF EXISTS index_name ON table_name;

Here,

- **index_name** is the name of the index that you want to drop.
- **table_name** is the name of the table that the index is associated with.

Example

In this example, let us try to drop an index in the SQL Server database.

Let us consider the previously created table CUSTOMERS and let us create an index for the NAME column in the table using the following query −

CREATE INDEX INDEX_NAME on CUSTOMERS(NAME);

Then, let us drop it using the following query −

DROP INDEX IF EXISTS INDEX_NAME ON CUSTOMERS;

Output

When we execute the above query, the output is obtained as follows −

Commands completed successfully.

Verification

Let's verify whether the index for the NAME is dropped or not using the following query −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

As you observe, the column NAME is deleted from the list of indexes.

index_name	index_description	index_keys
PK__CUSTOMER__3214EC27CB063BB7	clustered, unique, primary key locatedPRIMARY on PRIMARY	ID

Example

Now, let us delete an index **that doesn't exist** in the CUSTOMERS table using the following query −

DROP INDEX IF EXISTS INDEX_NAME ON CUSTOMERS;

Output

Since no indexes with the specified name exist in the database, so the above query simply terminates the execution without giving any error.

Commands completed successfully.

Removing indexes created by PRIMARY KEY or UNIQUE

The DROP INDEX statement does not drop indexes created by PRIMARY KEY or UNIQUE constraints. To drop indexes associated with them, we need to drop these constraints entirely. And it is done using the ALTER TABLE... DROP CONSTRAINT statement.

Syntax

Following is the syntax of the ALTER TABLE... DROP CONSTRAINT statement in SQL −

ALTER TABLE table_name DROP CONSTRAINT constraint_name;

Here,

- **table_name** is the name of the table that contains the PRIMARY KEY constraint.

- **constraint_name** is the name of the PRIMARY KEY constraint that you want to drop.

Example

Assume the previously created table (CUSTOMERS) and let us first list all the indexes that are created on the table using the following query −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

The list is displayed as follows −

index_name	index_description	index_keys
PK__CUSTOMER__3214EC27CB063BB7	nonclustered located on PRIMARYID	ID

Here, the **PK__CUSTOMER__3214EC27CB063BB7** is the name of the **PRIMARY KEY** constraint that was created on the **ID** column of the **CUSTOMERS** table.

Now, let us drop the index created by the PRIMARY KEY constraint.

ALTER TABLE customers

DROP CONSTRAINT PK__CUSTOMER__3214EC27CB063BB7;

Output

When we execute the above query, the output is obtained as follows −

Commands completed successfully.

Verification

Verify whether it is dropped or not by listing the existing indexes using the following query −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

The following error is displayed because the list of indexes is empty.

The object 'CUSTOMERS' does not have any indexes, or you do not have permissions.

The SQL Show Index Statement

The **SHOW INDEX** is the basic SQL statement to retrieve the information about the indexes that have been defined on a table. However, the SHOW INDEX statement only works on MySQL RDBMS and is not a valid statement in the SQL Server.

*To list the indexes created on a table in SQL Server, a system stored procedure **sp_helpindex** is used.*

The result-set obtained from querying the SHOW INDEX statement on a MySQL table contains the index information.

Syntax

Following is the syntax of the **SHOW INDEX** statement in MySQL –

SHOW INDEX FROM table_name;

Example

Following example demonstrates the working of SHOW INDEX statement in MySQL. First, create a table with the name CUSTOMERS in the MySQL database using the CREATE query below –

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (20, 2),
  PRIMARY KEY (ID)
);
```

Let us now insert some values into the above created table using the following query –

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', '32', 'Ahmedabad', 2000),
(2, 'Khilan', '25', 'Delhi', 1500),
```

(3, 'Kaushik', '23', 'Kota', 2000),

(4, 'Chaitali', '25', 'Mumbai', 6500),

(5, 'Hardik','27', 'Bhopal', 8500),

(6, 'Komal', '22', 'Hyderabad', 9000),

(7, 'Muffy', '24', 'Indore', 5500);

Once the data is inserted, create an index for the column **NAME** in the CUSTOMERS table using the following query −

CREATE INDEX INDEX_NAME ON CUSTOMERS(NAME);

Now, you can list all the indexes that are defined on the CUSTOMERS table using the following query −

SHOW INDEX FROM CUSTOMERS;

Output

On executing the above query, the output is displayed as follows −

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	1	index_name	1	NAME

Showing Indexes in SQL Server

In SQL server, the system stored procedure **sp_helpindex** is used to retrieve the information about the indexes that have been defined on a table. It returns the result as a table that contains detailed information about each index, including the name, type, and columns.

Syntax

Following is the basic syntax to list indexes defined on a table in SQL Server −

sp_helpindex [@objname =] 'name'

Here, **[@objname =] 'name'** specifies the name of the table for which the index information is being retrieved. The index information includes −

- **index_name** is the names of the columns that are included in index.
- **index_description** is the brief description of the index such as the type of index (like clustered or non-clustered).
- **index_keys** is the keys that are included in the index.

Example

CREATE INDEX INDEX_NAME on CUSTOMERS(NAME);

Now, let us list all the indexes that are created on the CUSTOMERS table using the system stored procedure **sp_helpindex** as shown below −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

Output

On executing the above query, the output is displayed as follows −

index_name	index_description	index_keys
INDEX_NAME	nonclustered located on PRIMARY	NAME
PK__CUSTOMER__ 3214EC27755869D9	clustered, unique, primary key located on PRIMARY	ID

SQL Unique Indexes

The SQL Unique Index ensures that no two rows in the indexed columns of a table have the same values (no duplicate values allowed).

A unique index can be created on one or more columns of a table using the CREATE UNIQUE INDEX statement in SQL.

Following are the points to be noted before creating a Unique Index on a table −

- ***If the unique index is only created on a single column, the rows in that column will be unique.***
- ***If a single column contains NULL in multiple rows, we cannot create a unique index on that column.***
- ***If the unique index is created on multiple columns, the combination of rows in these columns will be unique.***
- ***We cannot create a unique index on multiple columns if the combination of columns contains NULL in more than one row.***

Syntax

Following is the syntax for creating a **UNIQUE INDEX** in SQL −

CREATE UNIQUE INDEX index_name

ON table_name (column1, column2, ..., columnN);

Here,

- **index_name** is the name of the index that you want to create.
- **table_name** is the name of the table on which you want to create the index.
- **(column1, column2, ..., columnN)** are the names of one or more columns on which the unique index is being created.

First of all, let us create a table named **CUSTOMERS** using the following query −

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR(15) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS VARCHAR(25),
  SALARY DECIMAL(10, 4),
  PRIMARY KEY(ID)
);
```

Insert some values into the above-created table using the following query −

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', '32', 'Ahmedabad', 2000),
(2, 'Khilan', '25', 'Delhi', 1500),
(3, 'kaushik', '23', 'Kota', 2000),
(4, 'Chaitali', '26', 'Mumbai', 6500),
(5, 'Hardik','27', 'Bhopal', 8500),
(6, 'Komal', '22', 'Hyderabad', 9000),
(7, 'Muffy', '24', 'Indore', 5500);
```

Once the table is created, let us create a unique index for the column named **SALARY** in the CUSTOMERS table using the following query −

```
CREATE UNIQUE INDEX UNIQUE_ID ON CUSTOMERS (SALARY);
```

But, when we execute the above query, the output is obtained as follows −

```
ERROR 1062 (23000): Duplicate entry '2000.00' for key 'customers.UNIQUE_ID'
```

Since a unique index could not be created on SALARY column (due to duplicate values), let us create **Unique Index** on the **NAME** column of the same table, using the following query −

```
CREATE UNIQUE INDEX UNIQUE_ID ON CUSTOMERS (NAME);
```

Output

When we execute the above query, the output is obtained as follows −

Query OK, 0 rows affected (0.03 sec)

Records: 0 Duplicates: 0 Warnings: 0

Verification

Let's verify whether the unique index for the column NAME is created or not using the following query −

SHOW INDEX FROM CUSTOMERS;

As you observe the output below, you can find the column NAME along with the ID (PRIMARY KEY) in the list of indexes.

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	0	UNIQUE_ID	1	NAME

Updating with Duplicate Values

If we try to update the columns that have unique index with duplicate values, the database engine generates an error.

Example

Assume the previously created CUSTOMERS table and create a unique index on the column named **ADDRESS** using the following query −

CREATE UNIQUE INDEX ADD_UNIQUE_INDEX ON CUSTOMERS(ADDRESS);

Now, let us update the value in the column named ADDRESS with a duplicate (already existing data) value using the following query −

UPDATE CUSTOMERS SET ADDRESS = 'Mumbai' WHERE ADDRESS = 'Delhi';

Output

On executing the above query, the output is displayed as follows −

ERROR 1062 (23000): Duplicate entry 'Mumbai' for key 'customers.ADD_UNIQUE_INDEX'

Creating a unique index on Multiple Fields

We can also create a unique index on multiple fields or columns of a table using the CREATE UNIQUE INDEX statement. To do so, you just need to pass the name of the columns (you need to create the index on) to the query.

Example

Instead of creating a new table, let us consider the previously created CUSTOMERS table. We will create a unique index on the columns NAME and AGE using the following query –

CREATE UNIQUE INDEX MUL_UNIQUE_INDEX ON CUSTOMERS(NAME, AGE);

Output

When we execute the above query, the output is obtained as follows –

Query OK, 0 rows affected (0.04 sec)

Records: 0 Duplicates: 0 Warnings: 0.

Verification

Now, let us list all the indexes that are created on the CUSTOMERS table using the following query –

SHOW INDEX FROM CUSTOMERS;

As you observe you can find the column names NAME, and AGE along with the ID (PRIMARY KEY) in the list of indexes.

Table	Non_unique	Key_name	Seq_in_index	Column_name
customers	0	PRIMARY	1	ID
customers	0	MUL_UNIQUE_INDEX	1	NAME
customers	0	MUL_UNIQUE_INDEX	2	AGE

Data in a table is stored in the form of an unordered data structure called a "Heap", where rows are placed without any specific order. Thus, when retrieving data from a table, the query optimizer must scan the entire table to locate the requested rows. This process can be time-consuming, especially when we are dealing with large tables. To speed up the data retrieval, SQL provides a data object called index that stores and organizes table data in a specific way, allowing faster data access.

SQL Clustered Indexes

*A **clustered index** in SQL is a type of index that determines the physical order in which the data values will be stored in a table.*

When a clustered index is defined on a specific column, during the creation of a new table, the data is inserted into that column in a sorted order. This helps in faster retrieval of data since it is stored in a specific order.

- ***It is recommended to have only one clustered index on a table. If we create multiple clustered indexes on the same table, the table will have to store the same data in multiple orders which is not possible.***
- ***When we try to create a primary key constraint on a table, a unique clustered index is automatically created on the table. However, the clustered index is not the same as a primary key. A primary key is a constraint that imposes uniqueness on a column or set of columns, while a clustered index determines the physical order of the data in the table.***

MySQL database does not have a separate provisions for Clustered and Non-Clustered indexes. Clustered indexes are automatically created when PRIMARY KEY is defined on a table. And when the PRIMARY KEY is not defined, the first UNIQUE NOT NULL key is treated as a Clustered index.

Syntax

Following is the syntax to create a clustered index with SQL Server −

CREATE INDEX index_name ON table_name(column_name [asc|desc])

Where,

- **index_name**: specifies the name you want to give to the index being created.
- **column_name**: specifies the column(s) that will be indexed in the order specified.
- **asc|desc**: specifies the order (asc - ascending, desc - descending) in which the data should be sorted. The default sorting order is ascending order.

Example

In this example, let us create a clustered index on a table in SQL Server. For that, we need to first create a table named **CUSTOMERS** using the following query −

```
CREATE TABLE CUSTOMERS(
   ID INT NOT NULL,
   NAME VARCHAR (20) NOT NULL,
   AGE  INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (20, 2)
);
```

Now, insert some values into the CUSTOMERS table using the following query −

```
INSERT INTO CUSTOMERS VALUES
(7, 'Muffy', '24', 'Indore', 5500),
(1, 'Ramesh', '32', 'Ahmedabad', 2000),
```

(6, 'Komal', '22', 'Hyderabad', 9000),

(2, 'Khilan', '25', 'Delhi', 1500),

(4, 'Chaitali', '25', 'Mumbai', 6500),

(5, 'Hardik','27', 'Bhopal', 8500),

(3, 'Kaushik', '23', 'Kota', 2000);

The table is successfully created in the SQL Server database.

ID	NAME	AGE	ADDRESS	SALARY
7	Muffy	24	Indore	5500.00
1	Ramesh	32	Ahmedabad	2000.00
6	Komal	22	Hyderabad	9000.00
2	Khilan	25	Delhi	1500.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
3	Kaushik	23	Kota	2500.00

Now, let us create a clustered index on the column named **ID** using the following query –

CREATE CLUSTERED INDEX CLU_ID ON CUSTOMERS(ID ASC);

Output

On executing the above query, the output is displayed as follows –

Commands Completed Successfully.

Verification

To verify if the clustered index is defined on ID column, check whether the records of CUSTOMERS table are sorted by retrieving them using the following query –

```
SELECT * FROM CUSTOMERS;
```

The records of the table are sorted in ascending order based on values in the column named ID.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2500.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	9000.00
7	Muffy	24	Indore	5500.00

Creating Clustered Index on Multiple Columns

With the following example, let us understand how clustered index works when it is created on multiple columns of a table.

Instead of creating a new table, consider the previously created CUSTOMERS table and define a clustered index on multiple columns of this table, such as AGE and SALARY, using the following query −

CREATE CLUSTERED INDEX MUL_CLUS_ID ON CUSTOMERS (AGE, SALARY ASC);

Output

Verification

Now, let us verify whether the values in the columns **AGE** and **SALARY** is sorted or not −

SELECT * FROM CUSTOMERS;

As we can observe in the table below, the records are sorted only based on the values in AGE column and not with the values in SALARY column. So, it is recommended to have only one clustered index on a table.

ID	NAME	AGE	ADDRESS	SALARY
6	Komal	22	Hyderabad	9000.00
3	Kaushik	23	Kota	2500.00
7	Muffy	24	Indore	5500.00
2	Khilan	25	Delhi	1500.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
1	Ramesh	32	Ahmedabad	2000.00

SQL Non-Clustered Indexes

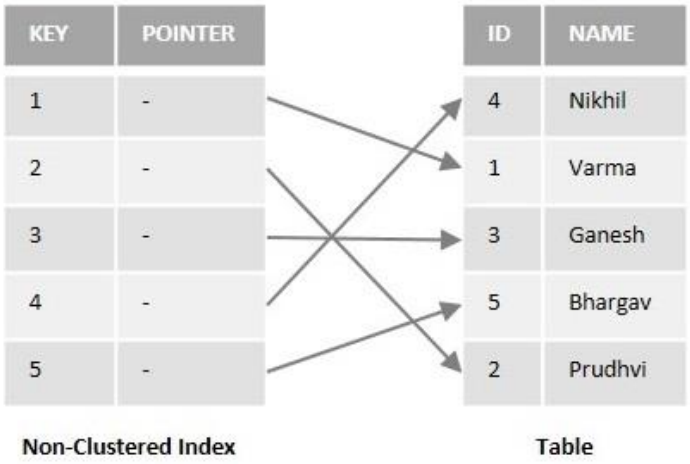
The SQL **Non-Clustered** index is similar to the Clustered index. When defined on a column, it creates a special table which contains the copy of indexed columns along with a pointer that refers to the location of the actual data in the table. However, unlike Clustered indexes, a Non-Clustered index cannot physically sort the indexed columns.

Following are some of the key points of the Non-clustered index in SQL –

- The non-clustered indexes are a type of index used in databases to speed up the execution time of database queries.
- These indexes require less storage space than clustered indexes because they do not store the actual data rows.
- We can create multiple non-clustered indexes on a single table.

MySQL does not have the concept of Non-Clustered indexes. The PRIMARY KEY (if exists) and the first NOT NULL UNIQUE KEY(if PRIMARY KEY does not exist) are considered clustered indexes in MySQL; all the other indexes are called Secondary Indexes and are implicitly defined.

To get a better understanding, look at the following figure illustrating the working of non-clustered indexes –



Assume we have a sample database table with two columns named **ID** and **NAME**. If we create a non-clustered index on a column named **ID** in the above table, it will store a copy of the ID column with a pointer that points to the specific location of the actual data in the table.

Syntax

Following is the syntax to create a **non-clustered** index in SQL Server −

CREATE NONCLUSTERED INDEX index_name ON table_name (column_name)

Here,

- **index_name**: holds the name of non-clustered index.
- **table_name**: holds the name of the table where you want to create the non-clustered index.
- **column_name**: holds the name of the column that you want to define the non-clustered index on.

Example

Let us create a table named **CUSTOMERS** using the following query −

```
CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE  INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (20, 2),
);
```

Let us insert some values into the above-created table using the following query −

```
INSERT INTO CUSTOMERS VALUES
(7, 'Muffy', '24', 'Indore', 5500),
(1, 'Ramesh', '32', 'Ahmedabad', 2000),
(6, 'Komal', '22', 'Hyderabad', 9000),
(2, 'Khilan', '25', 'Delhi', 1500),
(4, 'Chaitali', '25', 'Mumbai', 6500),
(5, 'Hardik','27', 'Bhopal', 8500),
```

(3, 'Kaushik', '23', 'Kota', 2000);

The table is successfully created in the SQL database.

ID	NAME	AGE	ADDRESS	SALARY
7	Muffy	24	Indore	5500.00
1	Ramesh	32	Ahmedabad	2000.00
6	Komal	22	Hyderabad	9000.00
2	Khilan	25	Delhi	1500.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
3	Kaushik	23	Kota	2500.00

Now, let us create a non-clustered index on a **single column** named **ID** using the following query −

CREATE NONCLUSTERED INDEX NON_CLU_ID ON customers (ID ASC);

Output

On executing the above query, the output is displayed as follows −

Commands Completed Successfully.

Verification

Let us retrieve all the indexes that are created on the CUSTOMERS table using the following query −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

As we observe, we can find the column named ID in the list of indexes.

	index_name	index_description	index_keys

1	NON_CLU_ID	nonclustered located on PRIMARY	ID

Now, retrieve the CUSTOMERS table again using the following query to check whether the table is sorted or not −

SELECT * FROM CUSTOMERS;

As we observe, the non-clustered index does not sort the rows physically instead, it creates a separate key-value structure from the table data.

ID	NAME	AGE	ADDRESS	SALARY
7	Muffy	24	Indore	5500.00
1	Ramesh	32	Ahmedabad	2000.00
6	Komal	22	Hyderabad	9000.00
2	Khilan	25	Delhi	1500.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
3	Kaushik	23	Kota	2500.00

Creating Non-Clustered Index on Multiple Columns

Instead of creating a new table, let us consider the previously created CUSTOMERS table. Now, try to create a non-clustered index on **multiple columns** of the table such as ID, AGE and SALARY using the following query −

CREATE NONCLUSTERED INDEX NON_CLUSTERED_ID ON CUSTOMERS (ID, AGE, SALARY);

Output

The below query will create three separate non-clustered indexes for ID, AGE, and SALARY.

Commands Completed Successfully.

Verification

Let us retrieve all the indexes that are created on the CUSTOMERS table using the following query −

EXEC sys.sp_helpindex @objname = N'CUSTOMERS';

As we observe, we can find the column names ID, AGE and SALARY columns in the list of indexes.

	index_name	index_description	index_keys
1	NON_CLU_ID	nonclustered located on PRIMARY	ID, AGE, SALARY

Types of Indexes

1. **Primary Key Index**: Automatically created when a primary key is defined.
2. **Unique Index**: Ensures all values in a column or a set of columns are unique.
3. **Full-Text Index**: Used for full-text searches.
4. **Spatial Index**: Used for spatial data types.
5. **Regular Index**: Improves the performance of queries on non-unique columns.

Syntax

The basic syntax for creating an index in MySQL is:

1)Creating an index:

CREATE INDEX index_name ON table_name (column1, column2, ...);

2)Creating a unique index:

CREATE UNIQUE INDEX index_name ON table_name (column1, column2, ...);

3)Dropping an index:

DROP INDEX index_name ON table_name;

Example

Let's go through an example where we create indexes on a table and then demonstrate their usage.

Full Example

1. **Create a Database**:

```
CREATE DATABASE company;
```

2. **Select the Database**:

```
USE company;
```

3. **Create the Employees Table**:

```
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    employee_code VARCHAR(10) UNIQUE,
    department_id INT
);
```

4. **Insert Data into the Employees Table**:

```
INSERT INTO employees (first_name, last_name, email, employee_code, department_id) VALUES
('Rahul', 'Sharma', 'rahul.sharma@example.com', 'E001', 1),
('Priya', 'Singh', 'priya.singh@example.com', 'E002', 2),
('Amit', 'Kumar', 'amit.kumar@example.com', 'E003', 1),
('Neha', 'Verma', 'neha.verma@example.com', 'E004', 3),
('Sahil', 'Mehta', 'sahil.mehta@example.com', 'E005', 2);
```

5. **Create an Index on the last_name Column**:

```
CREATE INDEX idx_last_name ON employees (last_name);
```

6. **Query the Employees Table:**

Let's perform a query to see how the index on the last_name column improves performance.

SELECT * FROM employees WHERE last_name = 'Singh';

Output

id	first_name	last_name	email	employee_code	department_id
2	Priya	Singh	priya.singh@example.com	E002	2

In this example, the index idx_last_name helps speed up the retrieval of rows where the last_name is 'Singh'.

Adding an Index to an Existing Table

Let's assume we have a table projects and we want to add an index to it.

1. **Create the Projects Table**:

```
CREATE TABLE projects (
    project_id INT PRIMARY KEY AUTO_INCREMENT,
    project_name VARCHAR(100) NOT NULL,
    project_code VARCHAR(10),
    department_id INT
);
```

2. **Add an Index to the Projects Table**:

CREATE INDEX idx_project_name ON projects (project_name);

3. **Insert Data into the Projects Table**:

```
INSERT INTO projects (project_name, project_code, department_id) VALUES
('Project A', 'P001', 1),
('Project B', 'P002', 2),
('Project C', 'P003', 1),
('Project D', 'P004', 3);
```

4. **Query the Projects Table**:

Let's perform a query to see how the index on the project_name column improves performance.

```
SELECT * FROM projects WHERE project_name = 'Project B';
```

Output

project_id	project_name	project_code	department_id
2	Project B	P002	2

In this example, the index idx_project_name helps speed up the retrieval of rows where the project_name is 'Project B'.

Dropping an Index

If an index is no longer needed, it can be dropped to save storage space and eliminate overhead on data modification operations.

Dropping the Index on the last_name Column:

```
DROP INDEX idx_last_name ON employees;
```

Dropping the Index on the project_name Column:

```
DROP INDEX idx_project_name ON projects;
```

Important Considerations

- **Performance Improvement**: Indexes significantly improve the performance of data retrieval operations, especially on large tables.
- **Storage Overhead**: Indexes require additional storage space. The more indexes a table has, the more storage it will consume.
- **Data Modification Overhead**: Indexes add overhead to data modification operations (INSERT, UPDATE, DELETE) because the index needs to be updated along with the data.
- **Index Selection**: Carefully select columns for indexing based on the most frequently used queries to get the best performance improvement.

SQL Transactions

A transaction is a unit or sequence of work that is performed on a database. Transactions are accomplished in a logical order, whether in a manual fashion by a user or automatically by some sort of a database program.

A transaction is the propagation of one or more changes to the database. For example, if you are creating, updating or deleting a record from the table, then you are performing a transaction on that table. It is important to control these transactions to ensure the data integrity and to handle database errors.

Practically, you will club many SQL queries into a group and you will execute all of them together as a part of a transaction.

Properties of Transactions

Transactions have the following four standard properties, usually referred to by the acronym **ACID**.

- **Atomicity** – ensures that all operations within the work unit are completed successfully. Otherwise, the transaction is aborted at the point of failure and all the previous operations are rolled back to their former state.
- **Consistency** – ensures that the database properly changes states upon a successfully committed transaction.
- **Isolation –** enables transactions to operate independently of and transparent to each other.
- **Durability** – ensures that the result or effect of a committed transaction persists in case of a system failure.

Transactional Control Commands

Transactional control commands are only used with the **DML Commands** such as - INSERT, UPDATE and DELETE. They cannot be used while creating tables or dropping them because these operations are automatically committed in the database. Following commands are used to control transactions.

- **COMMIT** – to save the changes.
- **ROLLBACK** – to roll back the changes.
- **SAVEPOINT** – creates points within the groups of transactions in which to ROLLBACK.
- **SET TRANSACTION** – Places a name on a transaction.

The COMMIT Command

The COMMIT command is the transactional command used to save changes invoked by a transaction. It saves all the transactions occurred on the database since the last COMMIT or ROLLBACK.

The syntax for the COMMIT command is as follows.

```
COMMIT;
```

Example

Firstly, let us create a table names CUSTOMERS using the following query –

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

We are inserting some records into the above-created table −

INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00),

(2, 'Khilan', 25, 'Delhi', 1500.00),

(3, 'Kaushik', 23, 'Kota', 2000.00),

(4, 'Chaitali', 25, 'Mumbai', 6500.00),

(5, 'Hardik', 27, 'Bhopal', 8500.00),

(6, 'Komal', 22, 'Hyderabad', 4500.00),

(7, 'Muffy', 24, 'Indore', 10000.00);

The table will be created as follows −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Following query would delete those records from the table which have AGE as 25 and then COMMIT the changes in the database.

DELETE FROM CUSTOMERS WHERE AGE = 25;

COMMIT;

Verification

The two rows from the table would be deleted and if you verify the contents of the CUSTOMERS table using the SELECT statement as −

SELECT * FROM CUSTOMERS;

The table will be displayed as follows −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
3	Kaushik	23	Kota	2000.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

The ROLLBACK Command

The ROLLBACK command is the transactional command used to undo transactions that have not already been saved to the database. This command can only undo transactions since the last COMMIT or ROLLBACK.

The syntax for a ROLLBACK command is as follows −

ROLLBACK;

Example

Consider the CUSTOMERS table having the following records −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00

4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Following query would delete those records from the table where the AGE value is 25 and then ROLLBACK the changes in the database.

```
DELETE FROM CUSTOMERS WHERE AGE = 25;

ROLLBACK;
```

Verification

The delete operation would not impact the table and the SELECT statement would produce the following result.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

The SAVEPOINT Command

A SAVEPOINT is a logical rollback point in a transaction.

Usually, when you execute the ROLLBACK command, it undoes the changes until the last COMMIT. But, if you create save points you can partially roll the transaction back to these points. You can create multiple save points between two commits.

The syntax to create a SAVEPOINT among the transactions is as shown below.

SAVEPOINT savepoint_name;

Then, to roll back to the SAVEPOINT created, you can use the following syntax −

ROLLBACK TO savepoint_name;

Example

Following is an example where you plan to delete the three different records from the CUSTOMERS table. You want to create a SAVEPOINT before each delete, so that you can ROLLBACK to any SAVEPOINT at any time to return the appropriate data to its original state.

Consider the CUSTOMERS table having the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

The following code block contains the series of operations.

```
SAVEPOINT SP1;
```

Query OK, 0 rows affected (0.00 sec)

```
DELETE FROM CUSTOMERS WHERE ID=1;
```

Query OK, 1 row affected (0.01 sec)

```
SAVEPOINT SP2;
```

Query OK, 0 rows affected (0.00 sec)

```
DELETE FROM CUSTOMERS WHERE ID=2;
```

Query OK, 0 rows affected (0.00 sec)

```
SAVEPOINT SP3;
```

Query OK, 0 rows affected (0.00 sec)

```
DELETE FROM CUSTOMERS WHERE ID=3;
```

Query OK, 1 row affected (0.01 sec)

Now that the three deletions have taken place, let us assume that you have changed your mind and decided to ROLLBACK to the SAVEPOINT that you identified as SP2. Because SP2 was created after the first deletion, the last two deletions are undone −

```
ROLLBACK TO SP2;
```

Verification

If you display the CUSTOMERS table, you can notice that only the first deletion took place since you rolled back to SP2.

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00

5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

The RELEASE SAVEPOINT Command

The RELEASE SAVEPOINT command is used to remove an existing SAVEPOINT.

The syntax for a RELEASE SAVEPOINT command is as follows.

RELEASE SAVEPOINT SAVEPOINT_NAME;

Once a SAVEPOINT has been released, you can no longer use the ROLLBACK command to undo transactions performed since the last SAVEPOINT.

The SET TRANSACTION Command

The SET TRANSACTION command can be used to initiate a database transaction. This command is used to specify characteristics for the transaction that follows. For example, you can specify a transaction to be read only or read write.

Syntax

The syntax for a SET TRANSACTION command is as follows.

SET TRANSACTION [READ WRITE | READ ONLY];

SQL Subqueries

An SQL **Subquery**, is a SELECT query within another query. It is also known as **Inner query** or **Nested query** and the query containing it is the outer query.

The outer query can contain the SELECT, INSERT, UPDATE, and DELETE statements. We can use the subquery as a column expression, as a condition in SQL clauses, and with operators like =, >, <, >=, <=, IN, BETWEEN, etc.

Rules to be followed

Following are the rules to be followed while writing subqueries −

- *Subqueries must be enclosed within parentheses.*
- *Subqueries can be nested within another subquery.*
- *A subquery must contain the SELECT query and the FROM clause always.*
- *A subquery consists of all the clauses an ordinary SELECT clause can contain: GROUP BY, WHERE, HAVING, DISTINCT, TOP/LIMIT, etc. However, an ORDER BY clause is only used when a TOP clause is specified. It can't include COMPUTE or FOR BROWSE clause.*
- *A subquery can return a single value, a single row, a single column, or a whole table. They are called scalar subqueries.*

Subqueries with the SELECT Statement

Subqueries are most frequently used with the SELECT statement. The basic syntax is as follows −

```
SELECT column_name [, column_name ]

FROM table1 [, table2 ]

WHERE  column_name

OPERATOR (SELECT column_name [,column_name ] FROM table1 [, table2 ]
[WHERE]);
```

Example

In the following query, we are creating a table named **CUSTOMERS** −

Open Compiler

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR(20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Here, we are inserting records into the above-created table using INSERT INTO statement −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00),
(2, 'Khilan', 25, 'Delhi', 1500.00),
(3, 'Kaushik', 23, 'Kota', 2000.00),
(4, 'Chaitali', 25, 'Mumbai', 6500.00),
(5, 'Hardik', 27, 'Bhopal', 8500.00),
(6, 'Komal', 22, 'Hyderabad', 4500.00),
(7, 'Muffy', 24, 'Indore', 10000.00);
```

The table is displayed as −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00

4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Now, let us check the following subquery with a SELECT statement.

Open Compiler

```
SELECT * FROM CUSTOMERS
WHERE ID IN (SELECT ID FROM CUSTOMERS WHERE SALARY > 4500);
```

This would produce the following result −

ID	NAME	AGE	ADDRESS	SALARY
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
7	Muffy	24	Indore	10000.00

Subqueries with the INSERT Statement

We can also use the subqueries along with the INSERT statements. The data returned by the subquery is inserted into another table.

The basic syntax is as follows −

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
   SELECT [ *|column1 [, column2 ] FROM table1 [, table2 ]
   [ WHERE VALUE OPERATOR ]
```

Example

In the following example, we are creating another table CUSTOMERS_BKP with similar structure as CUSTOMERS table −

Open Compiler

```
CREATE TABLE CUSTOMERS_BKP (
  ID INT NOT NULL,
  NAME VARCHAR(20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now to copy the complete records of CUSTOMERS table into the CUSTOMERS_BKP table, we can use the following query −

Open Compiler

```
INSERT INTO CUSTOMERS_BKP  SELECT * FROM CUSTOMERS
WHERE ID IN (SELECT ID FROM CUSTOMERS);
```

The above query produces the following output −

Query OK, 7 rows affected (0.01 sec)

Records: 7 Duplicates: 0 Warnings: 0

Verification

Using the SELECT statement, we can verify whether the records from CUSTOMERS table have been inserted into CUSTOMERS_BKP table or not −

Open Compiler

```
SELECT * FROM CUSTOMERS_BKP;
```

The table will be displayed as −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00

2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Subqueries with the UPDATE Statement

A subquery can also be used with the UPDATE statement. You can update single or multiple columns in a table using a subquery.

The basic syntax is as follows −

```
UPDATE table  SET column_name = new_value
[WHERE OPERATOR [VALUE](SELECT COLUMN_NAME FROM TABLE_NAME [WHERE]);
```

Example

We have the CUSTOMERS_BKP table available which is backup of CUSTOMERS table. The following example updates SALARY by 0.25 times in the CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

Open Compiler

```
UPDATE CUSTOMERS  SET SALARY = SALARY * 0.25
WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP WHERE AGE >= 27 );
```

Following is the output of the above query −

Query OK, 2 rows affected (0.01 sec)

Rows matched: 2 Changed: 2 Warnings: 0

Verification

This would impact two rows and if you verify the contents of the CUSTOMERS using the SELECT statement as shown below.

Open Compiler

SELECT * FROM CUSTOMERS;

The table will be displayed as –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	500.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	2125.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Subqueries with the DELETE Statement

The subquery can be used with the DELETE statement as well; like with any other statements mentioned above.

The basic syntax is as follows –

DELETE FROM TABLE_NAME

[WHERE OPERATOR [VALUE](SELECT COLUMN_NAME FROM TABLE_NAME)[WHERE)];

Example

We have a CUSTOMERS_BKP table available which is a backup of the CUSTOMERS table. The following example deletes the records from the

CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

Open Compiler

```
DELETE FROM CUSTOMERS
WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP WHERE AGE >= 27 );
```

The above query generate the following output −

OK, 2 rows affected (0.01 sec)

Verification

If you verify the contents of the CUSTOMERS table using the SELECT statement as shown below.

Open Compiler

```
SELECT * FROM CUSTOMERS;
```

The table will be displayed as −

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Preventing Duplicate Entries

To prevent the entry of duplicate records into a table, we can define a PRIMARY KEY or a UNIQUE Index on the relevant fields. These database constraints ensure that each entry in the specified column or set of columns is unique.

Example

Let us create a CUSTOMERS table using the following query −

```
CREATE TABLE CUSTOMERS (
  FIRST_NAME CHAR(20),
  LAST_NAME CHAR(20),
  SEX CHAR(10)
);
```

As we have not defined any constraints on the table, duplicate records can be inserted into it. To prevent such cases, add a PRIMARY KEY constraint on relevant fields (say LAST_NAME and FIRST_NAME together) −

```
ALTER TABLE CUSTOMERS ADD PRIMARY KEY (LAST_NAME, FIRST_NAME);
```

Using INSERT IGNORE Query:

Alternatively, we can use the INSERT IGNORE statement to insert records without generating an error for duplicates as shown below −

```
INSERT IGNORE INTO CUSTOMERS (LAST_NAME, FIRST_NAME) VALUES
( 'Jay', 'Thomas'),
( 'Jay', 'Thomas');
```

As you can see below, the table will only consist of a single record (ignoring the duplicate value).

FIRST_NAME	LAST_NAME	SEX
Thomas	Jay	NULL

Using REPLACE Query:

Or, use the REPLACE statement to replace duplicates as shown in the following query −

```
REPLACE INTO CUSTOMERS (LAST_NAME, FIRST_NAME) VALUES
( 'Ajay', 'Kumar'),
( 'Ajay', 'Kumar');
```

The table will contain the following records −

FIRST_NAME	LAST_NAME	SEX
Kumar	Ajay	NULL
Thomas	Jay	NULL

The choice between the INSERT IGNORE and REPLACE statements should be made based on the desired duplicate-handling behaviour. The INSERT IGNORE statement retains the first set of duplicate records and discards any subsequent duplicates. Conversely, the REPLACE statement preserves the last set of duplicates and erases any earlier ones.

Using UNIQUE Constraint:

Another way to enforce uniqueness in a table is by adding a UNIQUE constraint rather than a PRIMARY KEY constraint −

```
CREATE TABLE BUYERS (
  FIRST_NAME CHAR(20) NOT NULL,
  LAST_NAME CHAR(20) NOT NULL,
  SEX CHAR(10),
  UNIQUE (LAST_NAME, FIRST_NAME)
);
```

Counting and Identifying Duplicates

To count and identify duplicate records based on specific columns, we can use the COUNT function and GROUP BY clause.

Example

Following is the query to count duplicate records with FIRST_NAME and LAST_NAME in the BUYERS −

```
SELECT COUNT(*) as repetitions, LAST_NAME, FIRST_NAME

FROM BUYERS

GROUP BY LAST_NAME, FIRST_NAME

HAVING repetitions > 1;
```

This query will return a list of all the duplicate records in the PERSON_TABLE table. To identify sets of values that are duplicated, follow the steps given below −

- Determine which columns contain the values that may be duplicated.
- List those columns in the column selection list, along with the COUNT(*).
- List the columns in the GROUP BY clause as well.
- Add a HAVING clause that eliminates the unique values by requiring the group counts to be greater than one.

Eliminating Duplicates from a Table

We can use the DISTINCT keyword along with the SELECT statement to retrieve unique records from a table.

```
SELECT DISTINCT LAST_NAME, FIRST_NAME

FROM BUYERS

ORDER BY LAST_NAME;
```

Alternatively, you can include a GROUP BY clause specifying the columns you are selecting to eliminate duplicates −

```
SELECT LAST_NAME, FIRST_NAME
FROM BUYERS
GROUP BY LAST_NAME, FIRST_NAME;
```

Sequences in MySQL

In MySQL, we use the AUTO_INCREMENT attribute to generate unique integer values (sequences) for a column. By default, the sequence starts with an initial value of 1 and increments by 1 for each new row.

Syntax

Following is the syntax of AUTO_INCREMENT attribute in MySQL −

```
CREATE TABLE table_name (
        column1 datatype AUTO_INCREMENT,
        column2 datatype,
        column3 datatype,
        ...
        columnN datatype
);
```

In the following example, we are creating a table named CUSTOMERS. In addition to that, we are defining AUTO_INCREMENT on ID column of the table.

```
CREATE TABLE CUSTOMERS (
  ID INT AUTO_INCREMENT,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID));
```

Here, we are adding some records into the above created table −

INSERT INTO CUSTOMERS VALUES

(NULL, 'Ramesh', 32, 'Ahmedabad', 2000.00),

(NULL, 'Khilan', 25, 'Delhi', 1500.00),

(NULL, 'Kaushik', 23, 'Kota', 2000.00),

(NULL, 'Chaitali', 25, 'Mumbai', 6500.00),

(NULL, 'Hardik', 27, 'Bhopal', 8500.00),

(NULL, 'Komal', 22, 'Hyderabad', 4500.00),

(NULL, 'Muffy', 24, 'Indore', 10000.00);

The table will be created as follows −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

As we can see in the above table, the values in the ID column are auto incremented.

Starting a Sequence at a Particular Value in MySQL

By default, MySQL sequences start from 1. To start a sequence with a different value, we use the AUTO_INCREMENT in combination with the **ALTER** statement.

Syntax

Following is the syntax to start the sequence with different value −

```
ALTER TABLE table_name AUTO_INCREMENT = value;
```

In the following query, we are creating a table named BUYERS with AUTO_INCREMENT defined on the ID column.

```
CREATE TABLE BUYERS (
  ID INT AUTO_INCREMENT,
  NAME VARCHAR(20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Here, we are making the sequence start with 100 using the ALTER Statement as shown below −

```
ALTER TABLE BUYERS AUTO_INCREMENT=100;
```

Now, we are adding records into the BUYERS table using the INSERT INTO statement −

```
INSERT INTO BUYERS (NAME, AGE, ADDRESS, SALARY) VALUES
('Ramesh', 32, 'Ahmedabad', 2000.00),
('Khilan', 25, 'Delhi', 1500.00),
('Kaushik', 23, 'Kota', 2000.00),
('Chaitali', 25, 'Mumbai', 6500.00),
('Hardik', 27, 'Bhopal', 8500.00),
```

('Komal', 22, 'Hyderabad', 4500.00),

('Muffy', 24, 'Indore', 10000.00);

The table will be created as −

ID	NAME	AGE	ADDRESS	SALARY
100	Ramesh	32	Ahmedabad	2000.00
101	Khilan	25	Delhi	1500.00
102	Kaushik	23	Kota	2000.00
103	Chaitali	25	Mumbai	6500.00
104	Hardik	27	Bhopal	8500.00
105	Komal	22	Hyderabad	4500.00
106	Muffy	24	Indore	10000.00

As observed in the table above, the values in the "ID" column begin with 100 instead of 1.

Sequences in SQL Server

In SQL server, a sequence can be created using the **CREATE SEQUENCE** statement. The statement specifies the name of the sequence, the starting value, the increment, and other properties of the sequence.

Syntax

Following is the syntax to create a sequence in SQL −

CREATE SEQUENCE Sequence_Name

START WITH Initial_Value

INCREMENT BY Increment_Value

MINVALUE Minimum_Value

MAXVALUE Maximum_Value

CYCLE|NOCYCLE;

Here,

- **Sequence_Name** − This specifies the name of the sequence.
- **Initial_Value** − This specifies the starting value from where the sequence should start.
- **Increment_Value** − This specifies the value by which the sequence will increment by itself. This can be valued positively or negatively.
- **Minimum_Value** − This specifies the minimum value of the sequence.
- **Maximum_Value** − This specifies the maximum value of the sequence.
- **Cycle** − When the sequence reaches its Maximum_Value, it starts again from the beginning.
- **Nocycle** − An exception will be thrown if the sequence exceeds the Maximum_Value.

Example

First of all, let us create a table named CUSTOMERS using the following query −

```
CREATE TABLE CUSTOMERS (
  ID INT,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
);
```

We are inserting some records in the above-created table using INSERT INTO statement as shown in the query below −

```
INSERT INTO CUSTOMERS VALUES
(NULL, 'Ramesh', 32, 'Ahmedabad', 2000.00),
(NULL, 'Khilan', 25, 'Delhi', 1500.00),
```

(NULL, 'Kaushik', 23, 'Kota', 2000.00),

(NULL, 'Chaitali', 25, 'Mumbai', 6500.00),

(NULL, 'Hardik', 27, 'Bhopal', 8500.00),

(NULL, 'Komal', 22, 'Hyderabad', 4500.00),

(NULL, 'Muffy', 24, 'Indore', 10000.00);

The table is successfully created in the SQL database.

ID	NAME	AGE	ADDRESS	SALARY
NULL	Ramesh	32	Ahmedabad	2000.00
NULL	Khilan	25	Delhi	1500.00
NULL	Kaushik	23	Kota	2000.00
NULL	Chaitali	25	Mumbai	6500.00
NULL	Hardik	27	Bhopal	8500.00
NULL	Komal	22	Hyderabad	4500.00
NULL	Muffy	24	Indore	10000.00

Now, create a sequence using the following query −

CREATE SEQUENCE My_Sequence AS INT

START WITH 1

INCREMENT BY 1

MINVALUE 1

MAXVALUE 7

CYCLE;

In the above query, the sequence is named **My_Sequence** and it starts with the value 1 and increments by 1 each time a value is generated. The sequence has

a maximum value of 5 and cycles back to the starting value when it reaches the maximum value.

Once the sequence is created, it can be used to generate unique integer values. Now, let us update the data in the **ID** column of the CUSTOMERS table using the following query −

```
UPDATE CUSTOMERS SET ID = NEXT VALUE FOR my_Sequence;
```

Output

When you execute the above query, the output is obtained as follows −

(7 rows affected)

Verification

Let us verify whether is sequence is updated in the ID column of the table or not using the following query −

```
SELECT * FROM CUSTOMERS;
```

The table will be displayed as −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Auto Increment in MySQL

In MySQL, *you can add the auto-increment feature to a column of a table using the attribute named* ***AUTO_INCREMENT****.*

By default, when we define the AUTO_INCREMENT attribute on a column, the unique values are generated from "1"; and for each new record we enter into the table, the values in the column will increment by 1. Thus, the first record inserted will have a value of 1, the second record will have a value of 2, and so on.

Syntax

Following is the syntax to add AUTO_INCREMENT attribute to a column of a table in MySQL −

```
CREATE TABLE table_name(
   column1 datatype AUTO_INCREMENT,
   column2 datatype,
   column3 datatype,
   .....
   columnN datatype
);
```

Example

In the query to we are creating a table named CUSTOMERS and adding the AUTO_INCREMENT to the column named ID −

```
CREATE TABLE CUSTOMERS(
   ID INT PRIMARY KEY AUTO_INCREMENT,
   NAME VARCHAR (20) NOT NULL,
   AGE INT NOT NULL,
   ADDRESS CHAR (25),
   SALARY DECIMAL (18, 2)
);
```

Now, let us insert values into the CUSTOMERS table using the INSERT statement –

INSERT INTO CUSTOMERS (NAME, AGE, ADDRESS, SALARY) VALUES

("Ramesh", 32, "Ahmedabad", 2000.00),

("Khilan", 25, "Delhi", 1500.00),

("Kaushik", 23, "Kota", 2000.00),

("Chaitali", 25, "Mumbai", 6500.00);

Verification

To verify this, you need to retrieve the contents of the CUSTOMERS using the SELECT query as –

SELECT * FROM CUSTOMERS;

Output

Following is the output of the above query, here you can observe that the ID values are generated automatically –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00

AUTO_INCREMENT on Existing Columns

MySQL also allows you to implement the AUTO_INCREMENT attribute on an existing table, using the ALTER TABLE statement.

Following query starts incrementing the ID values from 5 in the CUSTOMERS table CUSTOMERS −

ALTER TABLE CUSTOMERS AUTO_INCREMENT = 100;

Now, let us insert more records to see if the ID values are auto incremented.

INSERT INTO CUSTOMERS(NAME, AGE, ADDRESS, SALARY) VALUES

("Hardik", 27, "Bhopal", 8500.00),

("Komal", 22, "MP", 4500.00),

("Muffy", 24, "Indore", 10000.00);

To view the above table data, we use the following SELECT query −

SELECT * FROM CUSTOMERS;

The output of the above query is shown below. It shows the auto increment in action. We are getting the ID values of the newly inserted records begins at 100.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
100	Hardik	27	Bhopal	8500.00
101	Komal	22	Hyderabad	4500.00

102	Muffy	24	Indore	10000.00

Cursors in SQL

*A **Cursor** is a temporary memory that is allocated by the database server at the time of performing the **D**ata **M**anipulation **L**anguage operations on a table, such as INSERT, UPDATE and DELETE etc. It is used to retrieve and manipulate data stored in the SQL table.*

In MySQL, you cannot declare a cursor directly outside of a stored procedure or function. Cursors are generally declared within stored procedures, functions, or blocks of SQL code in MySQL database.

Using cursors, we can perform multiple operations on each row of a result set, with or without returning the original data.

Properties of Cursors

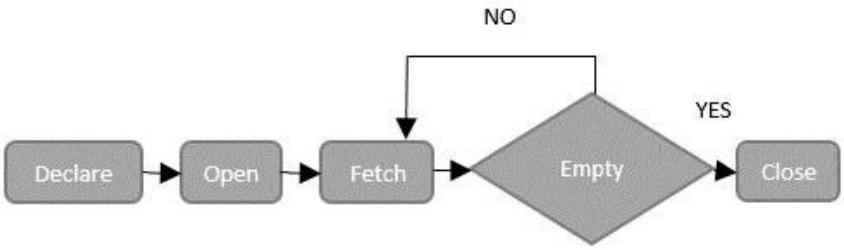
Following are the properties of MySQL Cursors −

- **READ ONLY –** We cannot update or modify any records in the table using the MySQL cursors. We can just fetch and process data from a table.
- **Non-Scrollable –** We can retrieve records from a table in a single direction, i.e. from the first record or the last. We cannot move backward or jump to a specific position within the result set.
- **Asensitive Cursor –** An asensitive cursor operates directly on the actual data in the database, it does not create a copy of the data. If any change is made to the data by other connections, it can affect the data that the cursor is working with.

*In addition to the Asensitive cursor there is another type known as **Insensitive Cursor**. An insensitive cursor uses a temporary copy of the data. Therefore, these cursors are insensitive (not affected) to the changes that are made in the table.*

Life Cycle of the Cursor

There are four steps to manage these cursors. Following diagram illustrates the lifecycle of an SQL cursor −



Now, let us discuss the phases of life cycle of the cursor one-by-one.

Declare Cursor Statement

In MySQL we can declare a cursor using the **DECLARE** statement and associate it with a SELECT statement to retrieve records from a database table.

However, this SELECT statement associated with a cursor does not use the INTO clause, as it's purpose is to fetch and process rows rather than assigning values to variables.

Syntax

Following is the syntax to declare a cursor in MySQL database −

DECLARE cursor_name CURSOR FOR select_statement;

Open Cursor Statement

After declaring a cursor in MySQL, the next step is to open the cursor using the OPEN statement. It initializes the result-set, allowing us to fetch and process rows from the associated SELECT statement in the cursor.

Syntax

Following is the syntax to open a cursor in MySQL database −

OPEN cursor_name;

Fetch Cursor Statement

Then, we can use the FETCH statement to retrieve the current row pointed by the cursor, and with each FETCH, the cursor moves to the next row in the result set. This allows us to process each row one by one.

Syntax

Following is the syntax to fetch a cursor in MySQL database −

FETCH cursor_name INTO variable_list;

Close Cursor Statement

Once all the rows are fetched, we must close the cursor to release the memory associated with it. We can do this using the CLOSE statement.

Syntax

Following is the syntax to close a cursor in MySQL database −

CLOSE cursor_name;

Example

In this example, let us see how to manage a cursor in a stored procedure.

Assume we have created a table with the name **CUSTOMERS** using the **CREATE TABLE** statement as follows −

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Now, let us insert some records into the **CUSTOMERS** table using the **INSERT** statement as follows −

```
INSERT INTO CUSTOMERS VALUES

(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),

(2, 'Khilan', 25, 'Delhi', 1500.00 ),

(3, 'Kaushik', 23, 'Kota', 2000.00 ),

(4, 'Chaitali', 25, 'Mumbai', 6500.00 );
```

Now, we will create a backup table named 'CUSTOMERS_BACKUP' to store customer data −

```
CREATE TABLE CUSTOMERS_BACKUP (

  ID INT NOT NULL,

  NAME VARCHAR (20) NOT NULL,

  PRIMARY KEY (ID)

);
```

Here, we are creating a stored procedure named **FetchCustomers** to fetch customer names from the **CUSTOMERS** table and inserting them one by one into the **BACKUP** table. We are using a cursor to iterate through the rows and a handler to detect the end of the result-set, ensuring all names are processed −

```
DELIMITER //
CREATE PROCEDURE FetchCustomers()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE customer_id INT;
  DECLARE customer_name VARCHAR(255);
  DECLARE auto_id INT;
  -- Declare cursor
  DECLARE MY_CURSOR CURSOR FOR SELECT id, name FROM CUSTOMERS;
  -- Declare exit handler
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  -- Open cursor
  OPEN MY_CURSOR;
  -- Fetch and insert rows
  read_loop: LOOP
    FETCH MY_CURSOR INTO customer_id, customer_name;
    IF done = 1 THEN
      LEAVE read_loop;
    END IF;
    -- Insert the fetched data into the backup table
    INSERT INTO customers_backup VALUES (customer_id, customer_name);
    -- Get the last auto-generated ID used in the insertion
    SET auto_id = LAST_INSERT_ID();
  END LOOP;
   -- Close cursor
   CLOSE MY_CURSOR;
END //
DELIMITER ;
```

Once we create the procedure successfully, we can execute it using the CALL statement as shown below −

```
CALL FetchCustomers();
```

Verification

You can verify the contents of the CUSTOMERS_BACKUP table using the SELECT statement as shown below −

```
SELECT * FROM CUSTOMERS_BACKUP;
```

The contents of the table would be −

ID	NAME
1	Ramesh
2	Khilan
3	Kaushik
4	Chaitali

SQL Database Tuning

Database Tuning in SQL is a set of activities performed to optimize a database and prevents it from becoming a bottleneck.

There are various techniques with which you can configure the optimal performance of a particular database. Database tuning overlaps with query tuning; so, good indexing and avoiding improper queries help in increasing the database efficiency. In addition, increasing storage, updating to latest database versions and investing in a more powerful CPU (if needed) are also some of the general techniques.

Database Tuning Techniques

We can implement the following techniques to optimize the performance of a database –

Database Normalization

Normalization is the process of removing of duplicate data from a database. We can normalize a database by breaking down larger tables into smaller related tables. This increases the performance of database as it requires less time to retrieve data from small tables instead of one large table.

Proper Indexes

In SQL, indexes are the pointers (memory address) to the location of specific data in database. We use indexes in our database to reduce query time, as the database engine can jump to the location of a specific record using its index instead of scanning the entire database.

Avoid Improper Queries

Choosing the correct query to retrieve data efficiently also improves the performance of a database. For example, choosing to retrieve an entire table when we only need the data in a single column will unnecessarily increase query time. So, query the database wisely.

Let us discuss some of the common improper queries made and how to rectify them to optimize the database performance.

1. Use SELECT *fields* instead of SELECT (*)

In large databases, we should always retrieve only the required columns from the database instead of retrieving all the columns, even when they are not needed. We can easily do this by specifying the column names in the SELECT statement instead of using the SELECT (*) statement.

Example

Assume we have created a table with name CUSTOMERS in MySQL database using CREATE TABLE statement as shown below –

Open Compiler

```
CREATE TABLE CUSTOMERS (
  ID INT NOT NULL,
```

```
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25),
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
);
```

Following query inserts values into this table using the INSERT statement −

Open Compiler

```
INSERT INTO CUSTOMERS VALUES
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00 ),
(2, 'Khilan', 25, 'Delhi', 1500.00 ),
(3, 'Kaushik', 23, 'Kota', 2000.00 ),
(4, 'Chaitali', 25, 'Mumbai', 6500.00 ),
(5, 'Hardik', 27, 'Bhopal', 8500.00 ),
(6, 'Komal', 22, 'Hyderabad', 4500.00 ),
(7, 'Muffy', 24, 'Indore', 10000.00 );
```

Let us say we only want the data in ID, NAME and SALARY columns of the CUSTOMERS table. So, we should only specify those three columns in our SELECT statement as shown below −

Open Compiler

```
SELECT ID, NAME, SALARY FROM CUSTOMERS;
```

Output

The output obtained is as shown below −

ID	NAME	SALARY
1	Ramesh	2000.00

2	Khilan	1500.00
3	Kaushik	2000.00
4	Chaitali	6500.00
5	Hardik	8500.00
6	Komal	4500.00
7	Muffy	10000.00

2. Use Wildcards

Wildcards (%) are characters that we use to search for data based on patterns. These wildcards paired with indexes only improves performance because the database can quickly find the data that matches the pattern.

Example

If we want to retrieve the names of all the customers starting with K from the CUSTOMERS table, then, the following query will provide the quickest result −

Open Compiler

```
SELECT ID, NAME FROM CUSTOMERS WHERE NAME LIKE 'K%';
```

Output

Following is the output of the above query −

ID	NAME
2	Khilan
3	Kaushik
6	Komal

3. Use Explicit Join

SQL JOINs are used to combine two tables based on a common column. There are two ways of creating a JOIN implicit join and explicit join. Explicit Join notation use the JOIN keyword with the ON clause to join two tables while the implicit join notation does not use the JOIN keyword and works with the WHERE clause.

Performance wise, they are both on the same level. However, in more complicated cases, the implicit join notation might produce completely different results than intended. Therefore, Explicit Joining is preferred.

4. Avoid using SELECT DISTINCT

The DISTINCT operator in SQL is used to retrieve unique records from the database. And on a properly designed database table with unique indexes, we rarely use it.

But, if we still have to use it on a table, using the GROUP BY clause instead of the DISTINCT keyword shows a better query performance (at least in some databases).

5. Avoid using Multiple OR

The OR operator is used to combine multiple conditions when filtering a database. Whenever we use OR in a filter condition each statement is processed separately. This degrades database performance as the entire table must be scanned multiple times to retrieve the data that matches the filter condition.

Instead, we can use a more optimized solution; by breaking the different OR conditions into separate queries, which can be processed parallelly by the database. Then, the results from these queries can be combined using UNION.

Example

For example, let us say we have a requirement of getting the details of all the customers whose age is greater than 25 or whose salary is greater than 2,000. The optimized query would be as show below −

Open Compiler

```
SELECT ID, NAME FROM CUSTOMERS WHERE AGE > 25
UNION
SELECT ID, NAME FROM CUSTOMERS WHERE SALARY > 2000;
```

Output

After executing the above code, we get the following output −

ID	NAME
1	Ramesh
5	Hardik
4	Chaitali
6	Komal
7	Muffy

6. Use WHERE instead of HAVING

The WHERE and HAVING clause are both used to filter data in SQL. However, WHERE clause is more efficient than HAVING. With WHERE clause, only the records that match the condition are retrieved. But with HAVING clause, it first retrieves all the records and then filters them based on a condition. Therefore, the WHERE clause is preferable.

Database Defragmentation

When the data is stored in a database, they are placed in contiguous physical locations. In this case, both logical locations and physical locations are in the same order.

However, when the database tables are altered by deleting or updating the records, indexes are also altered to suit the changes made. This will cause the indexes to be scattered across the storage. Physical locations also lose the contiguous allocation. Thus, reducing the database performance.

Defragmentation is the solution this problem. It will reorganize/rebuild the logical ordering of the indexes to match the physical ordering. But, this process first analyses the indexes and chooses whether they only need to be reorganized or rebuilt completely.

Built-In Tuning Tools

Some databases provide built-in tuning tools to monitor the database performance. For instance, the Oracle database provides the following tuning tools –

- **EXPLAIN –** In SQL, the EXPLAIN command give us the order in which a query is executed along with the estimated cost of each step. We can use this to find the query the least cost to optimize the database.
- **tkprof –** tkprof is a command that gives us various statistics, such as CPU and I/O usage of a query. By using these statistics, we can tune our queries to reduce CPU and I/O utilization to increase the efficiency of our database.